Tab 1

🧪 Lab3 — MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

A Cisco Networking Academy–style lab that blends classical networking with application-layer engineering. You will configure inter-VLAN routing, deploy a MIME-aware socket server, capture packets, and analyze how meaning flows through the stack.

📘 Lab Overview

Scenario

A client in one VLAN transfers files to a server in another VLAN using a custom MIME-based protocol (JSON header + binary payload). You will verify connectivity, observe encapsulation, and analyze packets using Wireshark.

This lab reinforces how Layer 2 segmentation, Layer 3 routing, TCP reliability, and application metadata interact.

🎯 Objectives

After completing this lab, you will be able to:

- Configure Router-on-a-Stick inter-VLAN routing
- Configure VLANs and trunking on a switch
- Deploy and run a MIME socket server and client
- Capture traffic using Wireshark
- Identify ARP, VLAN tags, TCP handshake, and application data
- Decode JSON headers in packet payload
- Troubleshoot network and application failures
- Document protocol behavior across OSI layers

🧰 Required Resources

- 1 Router (e.g., Cisco Systems ISR/virtual router)
- 1 Switch (Layer 2)
- 2 PCs or VMs
- Ethernet cables or virtual links

- Python 3 installed
- MIME file transfer project
- Wireshark installed (from Wireshark Foundation)

🗺️ Topology

```
None
PC1 (VLAN 10) ----\
                    \ access
                     \
                     [S1]---- trunk ----[R1]
                     /
                    / access
PC2 Server (VLAN 20)
```

📋 Addressing Scheme

Device	Interface	IP	Gateway
R1	G0/0.10	192.168.10.1/24	—
R1	G0/0.20	192.168.20.1/24	—
PC1-Client	NIC	192.168.10.10	192.168.10.1
PC2-Server	NIC	192.168.20.20	192.168.20.1

🧠 Background Theory

Router-on-a-Stick

A single physical interface carries multiple VLANs via 802.1Q tagging. Subinterfaces perform routing between broadcast domains.

MIME Metadata

MIME labels content type (e.g., image/png). Your protocol wraps files with JSON describing:

- MIME type
- Size

Why This Matters

Real systems — APIs, storage services, message brokers — depend on this exact pattern: structured metadata + binary payload over reliable transport.

🔧 Part 1 — Configure Network Devices

Step 1 — Router Configuration

```None
enable
configure terminal
hostname R1

interface g0/0/0
no shutdown

interface g0/0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0

end
write memory
```

```
R1(config)#
R1(config)#end
R1#write memory
Building configuration...
[OK]
*Feb 18 11:08:08.561: %SYS-5-CONFIG_I: Configured from console by console
R1#
*Feb 18 11:08:10.847: %SYS-6-PRIVCFG_ENCRYPT_SUCCESS: Successfully encrypted pri
vate config file
*Feb 18 11:08:17.282: %LINK-3-UPDOWN: Interface GigabitEthernet0/0/0, changed st
ate to up
*Feb 18 11:08:18.284: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEth
ernet0/0/0, changed state to up
R1#enable
R1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#interface g0/0/0
R1(config-if)#no shutdown
R1(config-if)#
R1(config-if)#interface g0/0/0.10
R1(config-subif)#encapsulation dot1Q 10

%Configuration of multiple subinterfaces of the same main
interface with the same VID (10) is not permitted.
This VID is already configured on GigabitEthernet0/0/0.1.

R1(config-subif)#ip address 192.168.10.1 255.255.255.0

% Configuring IP routing on a LAN subinterface is only allowed if that
subinterface is already configured as part of an IEEE 802.10, IEEE 802.1Q,
or ISL vLAN.

R1(config-subif)#
R1(config-subif)#interface g0/0/0.20
R1(config-subif)#encapsulation dot1Q 20

%Configuration of multiple subinterfaces of the same main
interface with the same VID (20) is not permitted.
This VID is already configured on GigabitEthernet0/0/0.2.

R1(config-subif)#ip address 192.168.20.1 255.255.255.0

% Configuring IP routing on a LAN subinterface is only allowed if that
subinterface is already configured as part of an IEEE 802.10, IEEE 802.1Q,
or ISL vLAN.

R1(config-subif)#
R1(config-subif)#end
R1#write memory
Building configuration...
[OK]
*Feb 18 11:18:33.090: %SYS-5-CONFIG_I: Configured from console by console
R1#
*Feb 18 11:18:35.266: %SYS-6-PRIVCFG_ENCRYPT_SUCCESS: Successfully encrypted pri
vate config file
```

Step 2 — Switch Configuration

```
None
enable
configure terminal
hostname S1

vlan 10
name CLIENTS
```

```
vlan 20
name SERVERS

interface g0/1
switchport mode trunk

interface g0/2
switchport mode access
switchport access vlan 10

interface g0/3
switchport mode access
switchport access vlan 20

end
write memory
```

COM5 - PuTTY

```
 A1, ENCOR Skills Assessment, Scenario 1
A1>enable
A1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
A1(config)#hostname S1
S1(config)#vlan 10
S1(config-vlan)#name CLIENTS
S1(config-vlan)#
S1(config-vlan)#vlan 20
S1(config-vlan)#name SERVERS
```

```
S1(config)#interface g0/1
S1(config-if)#switchport mode trunk
S1(config-if)#
S1(config-if)#interface g0/2
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#
S1(config-if)#interface g0/3
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 20
S1(config-if)#
S1(config-if)#end
S1#write memory
Building configuration...
[OK]
S1#
Jan  1 04:05:28.458: %SYS-5-CONFIG_I: Configured from console by console
S1#
```

Step 3 — Configure Hosts

Set static IP addresses per table.

🔍 Part 2 — Verify Connectivity

From PC1:

```
None
ping 192.168.20.20
```

```
Pinging 192.168.20.20 with 32 bytes of data:

Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time=1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

Verify:

- Successful replies
- ARP entries

Router checks:

```
None
show ip route
show ip interface brief
```

```
R1>show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  administratively down down
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
R1>show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0.10
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0.10
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/0.20
L       192.168.20.1/32 is directly connected, GigabitEthernet0/0.20

R1>show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  administratively down down
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
R1>
```

Switch checks:

```
None
show interfaces trunk
show mac address-table
```

```
R1>show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  administratively down down
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
R1>show interfaces trunk
```

🧪 Part 3 — Deploy MIME Server

On Server (PC2)

```
None
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

On Client (PC1)

```
None
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port
65432
```

Confirm file saved.

📡 Part 4 — Wireshark Capture Walkthrough on PC1 and PC1

Step 1 — Start Capture

Open Wireshark.

Select interface connected to network.

Apply capture filter:

```
None
tcp port 65432
```

Step 2 — Initiate Transfer

Send file again.

Stop capture after completion.

Step 3 — Analyze Packets

A — ARP Exchange

Look for:

- Who has 192.168.20.20?

Confirm MAC resolution.

B — TCP Three-Way Handshake

Identify:

1 SYN 2 SYN-ACK 3 ACK

Check sequence numbers.

C — Data Transfer

Follow stream:

Right click → Follow → TCP Stream.

You should see JSON header similar to:

```
None
{"mime_type":"image/png","size":12345}
```

Followed by binary.

D — VLAN Tags

If capturing on trunk:

Look for:

```
None
802.1Q Virtual LAN
```

Note VLAN ID.

🔬 Packet Dissection Guide

Layer	Field	What to Observe
Ethernet	Src/Dst MAC	Frame direction
802.1Q	VLAN ID	Segmentation
IP	TTL	Routing behavior
TCP	Flags	Reliability
Payload	JSON	MIME metadata

🛠️ Part 5 — Troubleshooting Labs

Scenario 1 — Broken Routing

Shutdown subinterface.

Observe:

- Ping failure
- TCP timeout

Ethernet

- MAC ปลายทางเป็น MAC ของ Router-
- Frame ถูกส่งออกจาก Client

802.1Q

- VLAN ID ถูกต้อง (เช่น VLAN 10)
- แต่ Router ไม่ตอบกลับ

IP

- TTL ไม่ลด เพราะ packet ไม่ถูกส่งข้าม Router
- ไม่มี Reply กลับมา

TCP / ICMP

- Ping ไม่มี Echo Reply
- TCP ไม่มี SYN-ACK
- เกิด Timeout

Scenario 2 — Firewall Block

Block port.

Observe:

- SYN retransmissions

Ethernet

- Frame ถูกส่งถึง Server

IP

- TTL ปกติ
- Routing ทำงานปกติ

TCP

- เห็น SYN
- ไม่มี SYN-ACK
- SYN ถูกส่งซ้ำหลายครั้ง (Retransmission)

Payload

- ไม่มี JSON เพราะ Handshake ไม่สำเร็จ

Scenario 3 — Wrong Gateway

Observe:

- ARP incomplete

Ethernet

- เห็น ARP Broadcast: Who has 192.168.x.x?
- ไม่มี ARP Reply

802.1Q

- VLAN ถูกต้อง

IP

- ไม่มี TCP หรือ ICMP เลย
- เพราะยัง resolve MAC ไม่ได้

TCP

- ไม่เกิด Handshake

Scenario 4 — Packet Loss Simulation

Drop packets.

Observe:

- TCP retransmissions
- Window adjustment

Ethernet

- Frame ถูกส่งออกปกติ

IP

- TTL ลดลงตามปกติ

TCP

- มี Retransmission
- มี Duplicate ACK
- Window size เปลี่ยน (ลดลง)

Payload

- JSON ถูกส่งซ้ำบางส่วน

📝 Lab Report Requirements

Include:

- Screenshots of configurations
- Ping results
- Wireshark capture screenshots
- JSON header evidence
- Encapsulation explanation
- Troubleshooting notes

🎓 Assessment Questions

1. Why does TCP retransmit lost segments?

TCP เป็นโปรโตคอลแบบ **Connection-Oriented และ Reliable**
มีหน้าที่รับประกันว่าข้อมูลจะถูกส่งถึงปลายทางครบถ้วนและเรียงลำดับถูกต้อง

เมื่อมี segment สูญหายระหว่างทาง (เช่น จาก packet loss หรือ network congestion)
ปลายทางจะไม่ส่ง ACK สำหรับข้อมูลส่วนนั้น หรือผู้ส่งตรวจพบ timeout

ดังนั้น TCP จึง:

- ใช้ **Sequence Number** เพื่อตรวจสอบลำดับข้อมูล
- ใช้ **ACK (Acknowledgment)** เพื่อยืนยันการรับข้อมูล
- ใช้ **Retransmission Timer** เพื่อส่งซ้ำเมื่อไม่ได้รับ ACK

2. How does VLAN tagging affect broadcast domains?

VLAN (Virtual LAN) ทำหน้าที่แบ่งเครือข่าย Layer 2 ออกเป็นหลาย Broadcast Domain

เมื่อใช้ 802.1Q VLAN Tag:

- Frame จะถูกติด Tag ระบุ VLAN ID
- Switch จะส่ง frame ไปเฉพาะอุปกรณ์ใน VLAN เดียวกัน

ผลที่เกิดขึ้น:

3. Broadcast ใน VLAN 10 จะไม่กระจายไป VLAN 20
   TCP Header ทำหน้าที่ควบคุมการส่งข้อมูล เช่น:
- Source/Destination Port
- Sequence Number
- ACK Number
- Flags (SYN, ACK)

TCP ไม่สนใจว่า “ข้อมูลคืออะไร”
มันสนใจแค่ส่งให้ครบและถูกลำดับ

ในขณะที่ MIME (Multipurpose Internet Mail Extensions):

- เป็นข้อมูลระดับ Application Layer
- ระบุประเภทของข้อมูล เช่น image/png, application/json
- บอกขนาดไฟล์ และ metadata
4. ลดปริมาณ broadcast traffic
   Routing Decision เกิดขึ้นที่ **Layer 3 (Network Layer)**

Router จะ:

1. อ่าน Destination IP Address
2. ตรวจสอบ Routing Table
3. เลือกเส้นทางที่เหมาะสม
4. ส่ง packet ไปยัง Next Hop

Switch (Layer 2) ไม่ทำ routing
Switch ทำงานตาม MAC Address เท่านั้น

5. เพิ่มความปลอดภัยและการแยกเครือข่าย

ก่อนที่ TCP จะส่งข้อมูลได้
ต้องมีการสร้าง Ethernet Frame ก่อน

Ethernet Frame ต้องมี:

- Source MAC
- Destination MAC

แต่ TCP ทำงานด้วย IP Address
ดังนั้นจึงต้องใช้ ARP (Address Resolution Protocol) เพื่อ:

1. ถามว่า “ใครมี IP นี้?”
2. รับ MAC Address กลับมา
3. นำ MAC ไปใส่ใน Ethernet Frame

ลำดับที่เกิดขึ้นจริงคือ:

ARP → สร้าง MAC mapping → TCP Handshake → Data Transfer

6. What role does MIME play compared to TCP headers?

MIME (Multipurpose Internet Mail Extensions) ทำงานในระดับ **Application Layer (Layer 7)**
มีหน้าที่อธิบาย “ประเภทและความหมายของข้อมูล” ที่ถูกส่ง เช่น

- image/png
- application/json
- text/plain

MIME จะบอกว่าไฟล์ที่ส่งเป็นไฟล์ประเภทอะไร และมีขนาดเท่าไร
เพื่อให้ปลายทางรู้วิธีจัดการข้อมูลนั้นอย่างถูกต้อง

ในขณะที่ TCP Header ทำงานในระดับ **Transport Layer (Layer 4)**
ทำหน้าที่ควบคุมการรับส่งข้อมูล เช่น

-กำหนดพอร์ตต้นทาง/ปลายทาง

-ลำดับข้อมูล (Sequence Number)
-ตรวจสอบการรับข้อมูล (ACK)
-รับประกันความถูกต้องและความครบถ้วนของข้อมูล

7. Where does routing decision occur?

Routing Decision เกิดขึ้นที่ **Network Layer (Layer 3)**

เมื่อ Router ได้รับ Packet จะ:

1. อ่าน Destination IP Address
2. ตรวจสอบ Routing Table
3. เลือกเส้นทางที่เหมาะสมที่สุด
4. ส่ง Packet ไปยัง Next Hop

Switch (Layer 2) จะไม่ตัดสินใจเส้นทางโดยใช้ IP
Switch ใช้ MAC Address ในการส่งต่อ Frame เท่านั้น

ดังนั้นการตัดสินใจว่า Packet จะไปทางไหนในเครือข่าย
จะเกิดขึ้นที่ Router ใน Layer 3

8. Why does ARP precede TCP communication?

ก่อนที่ TCP จะส่งข้อมูลได้ จำเป็นต้องสร้าง Ethernet Frame ก่อน
ซึ่ง Ethernet Frame ต้องมี:

- Source MAC Address
- Destination MAC Address

แต่ TCP ทำงานโดยใช้ IP Address
จึงต้องมี ARP (Address Resolution Protocol) เพื่อแปลง IP Address ให้เป็น MAC Address

ลำดับเหตุการณ์จริงในเครือข่ายคือ:

1. เครื่องต้นทางรู้ IP ปลายทาง
2. ใช้ ARP เพื่อถามหา MAC Address
3. ได้ MAC Address แล้วจึงสร้าง Ethernet Frame
4. หลังจากนั้นจึงเริ่ม TCP 3-Way Handshake

ดังนั้น ARP ต้องเกิดก่อน เพราะการสื่อสารระดับ Layer 2 ต้องใช้ MAC Address ก่อนที่ TCP จะเริ่มทำงานได้

📊 Reflection Exercise

Explain:

> How does application meaning survive multiple encapsulation layers?

ในเครือข่ายคอมพิวเตอร์ ข้อมูลจาก Application Layer จะถูก “ห่อหุ้ม” (Encapsulation) ผ่านหลายชั้นของ OSI Model ได้แก่:

Application
→ Transport (TCP)
→ Network (IP)
→ Data Link (Ethernet)
→ Physical (สัญญาณไฟฟ้า/คลื่น)

แม้ว่าข้อมูลจะถูกห่อหุ้มหลายชั้น แต่ “ความหมาย” ของข้อมูลยังคงอยู่ เพราะแต่ละ Layer ทำหน้าที่เฉพาะของตนเอง และ **ไม่แก้ไขเนื้อหาของ Payload**

หลักการสำคัญที่ทำให้ความหมายไม่สูญหาย

1. **Layer Independence (การแยกหน้าที่ของแต่ละชั้น)**
   แต่ละ Layer ทำงานเฉพาะส่วนของตน เช่น:
   - TCP ดูแลลำดับและความถูกต้องของข้อมูล
   - IP ดูแลเส้นทางการส่ง
   - Ethernet ดูแล MAC Address
2. แต่ไม่มี Layer ใดแก้ไขเนื้อหาของข้อมูล Application
3. **Payload Transparency (ข้อมูลชั้นบนถูกมองเป็น Payload)**
   สำหรับ TCP, ข้อมูล JSON หรือไฟล์ภาพเป็นเพียง “ข้อมูลไบต์”
   TCP ไม่สนใจว่ามันคือรูปภาพหรือข้อความ
4. **Encapsulation และ Decapsulation**
   เมื่อข้อมูลถึงปลายทาง จะมีการ “แกะห่อ” (Decapsulation) ทีละชั้น
   จนเหลือเฉพาะข้อมูล Application เดิม
5. **End-to-End Principle**
   ความหมายของข้อมูลถูกสร้างและตีความเฉพาะที่ต้นทางและปลายทาง

อุปกรณ์กลางทาง (Router, Switch) ไม่แตะต้องเนื้อหา

⚠️ Common Pitfalls

- Server bound to localhost only
- Missing trunk
- Wrong VLAN
- OS firewall enabled
- Capture on wrong interface

🚀 Extension Activities

- Add TLS encryption
- Measure throughput vs latency
- Capture checksum errors
- Add checksum field to protocol
- Containerize server
- Compare with HTTP transfer

🧬 Instructor Notes — What This Lab Really Teaches

This is not just networking.

It demonstrates:

- Protocol layering philosophy
- Reliability engineering
- Metadata semantics
- Distributed system fundamentals

The student watches a file become:

intention → bytes → frames → signals → bytes → meaning

That transformation is the essence of computer networks.

TCP ping

Here's how you can modify the script to accept an IP address and port number as command-line arguments using sys.argv:

```python
import sys
from tcppinglib import tcpping
import asyncio

def main():
    # Check if correct number of arguments is provided
    if len(sys.argv) != 3:
        print("Usage: python script.py <IP_ADDRESS> <PORT>")
        sys.exit(1)

    ip_address = sys.argv[1]
    try:
        port = int(sys.argv[2])
    except ValueError:
        print("Error: Port must be an integer.")
        sys.exit(1)

    # Perform TCP ping
    host_stats = tcpping(ip_address, port=port, count=4, interval=1)

    print(f"Host: {host_stats.ip_address}")
    print(f"Is alive: {host_stats.is_alive}")
    print(f"Average RTT: {host_stats.avg_rtt:.2f} ms")
    print(f"Packet loss: {host_stats.packet_loss}%")

if __name__ == "__main__":
    main()
```

Explanation:

· The script uses sys.argv to read command-line arguments.
· It expects two arguments: the IP address (or hostname) and the port number.
· Basic error handling ensures both arguments are provided and the port is a valid integer.
· The tcpping function is called with the provided IP and port, keeping the same count and interval for demonstration.

You can run it like:

```bash
python script.py 192.168.1.1 80
```

If you also want to include the asynchronous version, you can adapt it similarly, but the synchronous example above covers the core requirement.

Role-based procedures

👥 Role-Based Lab Procedure: MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

🎭 Team Roles

Role Primary Responsibilities Suggested Tools
Network Infrastructure Engineer Configures router and switch, sets up VLANs and trunking, assigns IP addresses to router subinterfaces, verifies Layer 2/3 connectivity, troubleshoots network issues. Console access (PuTTY, SecureCRT), Cisco IOS CLI, ping, traceroute, show commands, possibly Packet Tracer for simulation.
Client/Server Engineer Deploys and runs the MIME server and client scripts on the two PCs, initiates file transfers, captures traffic with Wireshark, observes application behavior. Python 3, terminal/command prompt, Wireshark, the provided server/main_enhanced.py and client/main_enhanced.py scripts, a sample file (e.g., sample.png).
Knowledge Manager / Supervisor Oversees the lab workflow, documents configurations and results, leads analysis of packet captures, verifies understanding of concepts, coordinates troubleshooting, compiles the final report. Note-taking app (OneNote, Google Docs), Wireshark (for detailed analysis), lab manual, diagramming tool (draw.io), camera/screenshot tool.

📋 Step-by-Step Procedure

All three team members should be present and collaborate. The Knowledge Manager ensures every step is understood and documented.

Part 1 — Configure Network Devices

Network Infrastructure Engineer

· Access the router (R1) console and configure:
  · Hostname, interface G0/0, subinterfaces G0/0.10 and G0/0.20 with 802.1Q encapsulation and IP addresses per the addressing table.
  · Issue no shutdown on the physical interface.
  · Save configuration with write memory.
· Access the switch (S1) console and configure:
  · VLANs 10 and 20.
  · Interface G0/1 as trunk.
  · Interfaces Fa0/1 (access VLAN 10) and Fa0/2 (access VLAN 20).
  · Save configuration.
· Verify configurations with show ip interface brief, show interfaces trunk, show vlan brief.

Client/Server Engineer

· Not actively involved in this part, but observe the configuration process and take notes of the commands used (with help from the Knowledge Manager).

Knowledge Manager / Supervisor

· Document all configuration commands and their expected effects.
· Take screenshots of the running configurations.
· Ensure that the physical cabling matches the topology (PC1 connected to Fa0/1, PC2 to Fa0/2, router G0/0 to switch G0/1).
· Briefly explain the purpose of trunking and subinterfaces to the team.

Part 2 — Verify Connectivity

Network Infrastructure Engineer

· From the router, ping both PC IP addresses to confirm router-to-host connectivity:
```
ping 192.168.10.10
ping 192.168.20.20
```
· Use show ip route to confirm routes to both subnets are present (directly connected).

Client/Server Engineer

· On PC1, set the static IP address 192.168.10.10/24 with gateway 192.168.10.1.
· On PC2, set the static IP address 192.168.20.20/24 with gateway 192.168.20.1.
· From PC1, ping PC2:
```
ping 192.168.20.20
```
· If pings fail, work with the Network Infrastructure Engineer to troubleshoot (check VLAN assignment, trunk, IP settings).

Knowledge Manager / Supervisor

· Record successful ping results (screenshot).
· Verify that ARP entries are populated on both PCs (arp -a).
· Ensure the team understands why ping works across VLANs (router-on-a-stick provides inter-VLAN routing).

Part 3 — Deploy MIME Server and Transfer a File

Client/Server Engineer

· On PC2 (server) , open a terminal and start the server:
```bash
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```
· On PC1 (client) , open another terminal and send a file:
```bash
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port 65432
```
· Confirm that the file is saved on the server side (e.g., check the destination folder).
· If the transfer fails, verify that the server is listening on 0.0.0.0 (not localhost) and that no firewall is blocking port 65432.

Network Infrastructure Engineer

· Monitor network statistics during transfer (optional, e.g., show interface counters).
· Assist in troubleshooting connectivity if needed.

Knowledge Manager / Supervisor

· Document the exact commands used and their output.
· Ensure the team understands the MIME header concept (JSON metadata + binary payload).
· Take a screenshot of the server log showing the received file information.

Part 4 — Wireshark Capture and Analysis

Client/Server Engineer (both PC1 and PC2)

· On both PCs, launch Wireshark and start capturing on the network interface connected to the switch.
· Apply a capture filter to isolate the traffic:
```
tcp port 65432
```
· On PC1, re-run the file transfer command (the client).
· Stop the capture on both machines after the transfer completes.
· Save the capture files (e.g., client_capture.pcapng, server_capture.pcapng).

Knowledge Manager / Supervisor

· Lead the analysis of the captured packets.
  · On PC1's capture, locate:
    · ARP request/reply for the server's IP (if not already cached).
    · TCP three-way handshake (SYN, SYN-ACK, ACK).

  · Data packets carrying the JSON header (right-click → Follow → TCP Stream to see the full conversation).
  · The binary file data.
 · On PC2's capture, compare the same packets from the server's perspective.
· If capturing on the trunk link (optional), look for 802.1Q tags with VLAN IDs 10 and 20.
· For each layer (Ethernet, 802.1Q, IP, TCP, Application), note the fields listed in the dissection guide.
· Take screenshots of:
 · ARP packet details.
 · TCP handshake packets.
 · The JSON header inside the TCP stream.
 · A packet containing binary payload.
 · (Optional) VLAN tag information.

Network Infrastructure Engineer

· Assist in interpreting VLAN tags if trunk capture was performed.
· Explain how the router decides to forward packets based on destination IP and VLAN.

Part 5 — Troubleshooting Scenarios

The team works together to simulate failures and observe the effects. The Knowledge Manager predicts the expected behavior, then the group implements the fault and observes.

Scenario Action by … Expected Observation Documented Result
Broken Routing Network Infrastructure Engineer shuts down subinterface G0/0.10 on the router. Ping from PC1 to PC2 fails; TCP SYN from client times out (retransmissions). Screenshot of failed ping and Wireshark showing SYN retransmits.
Firewall Block Client/Server Engineer (on server) adds a firewall rule to block port 65432. Client's TCP SYN is unanswered; Wireshark shows repeated SYN attempts. Screenshot of firewall rule and capture showing no SYN-ACK.
Wrong Gateway Client/Server Engineer changes PC1's gateway to an incorrect IP. Ping fails; ARP requests for the gateway remain unanswered ("ARP incomplete"). Screenshot of arp -a showing incomplete entry.
Packet Loss Network Infrastructure Engineer uses a traffic control tool (e.g., tc on Linux) to drop 10% of packets. TCP retransmissions and possible window size adjustment; file still transfers reliably. Wireshark capture showing duplicate ACKs and retransmissions.

Knowledge Manager / Supervisor records each observation and leads a discussion on why TCP behaves the way it does.

📝 Lab Report Compilation

The Knowledge Manager is responsible for assembling the final lab report, incorporating:

· Screenshots from all parts (configs, ping results, Wireshark captures, JSON header).
· Explanations of each observation, linking to OSI layers.
· Answers to the assessment questions.
· A reflection on how application-layer meaning survives encapsulation.

The Network Infrastructure Engineer and Client/Server Engineer provide their notes, captures, and any troubleshooting logs.

✅ Collaboration Checkpoints

At the end of each major part, the team should pause and:

· Review what was accomplished.
· Ensure all documentation (screenshots, notes) is saved.
· Discuss any unexpected issues and how they were resolved.

This role-based approach ensures every student actively contributes while learning both the technical and analytical aspects of the lab.